DAY 07 | HANDS-ON ACTIVITIES

SQLAlchemy and API Docs: Problem Statements

Seven problems that grow the Library API. SQLite works the same in Colab and VS Code.

Prepared by **Ahmed** | linkedin.com/in/ahmed-al-imaad

How to work: read the problem, write the steps in plain English first, then write the code. Test with the sample run, then try your own inputs, including wrong ones.

Problem 1: Spot the Relationship

Level: Warm-up | **Concepts:** relationship design

For each pair, write the relationship type and where the foreign key or link table goes: customer and order, order and product, user and profile, doctor and patient, country and capital.

Sample run / expected result

```
customer - order: one-to-many, FK customer_id on orders
order - product: many-to-many, link table order_items
```

Hints

- Ask how many of each side the other can have.

Problem 2: Authors and Books

Level: Easy | **Concepts:** one-to-many, back_populates

Create Author and Book models with a one-to-many relationship. Add 3 authors with 2 books each, then print each author with their book titles.

Sample run / expected result

```
Ito: Tokyo Nights, Rain in Kyoto
Rao: Monsoon Code, Chai Stories
```

Hints

- relationship(back_populates=...) on both classes

Problem 3: Tags

Level: Medium | **Concepts:** many-to-many, association table

Add Tag and a book_tags link table. Tag books with genres, then print all books tagged "fiction" and all tags of one book.

Sample run / expected result

```
fiction: Tokyo Nights, Chai Stories
Tokyo Nights: fiction, travel
```

Hints

- select(Book).join(Book.tags).where(Tag.name == "fiction")

Problem 4: Constraints in Action

Level: Medium | **Concepts:** unique, nullable, IntegrityError

Make author name unique and book title not nullable. Try to break each rule and catch IntegrityError, rolling back each time.

Sample run / expected result

```
Duplicate author -> IntegrityError, rolled back
Missing title -> IntegrityError, rolled back
```

Hints

- from sqlalchemy.exc import IntegrityError

Problem 5: Safe Transfer

Level: Challenging | **Concepts:** transactions, rollback

Create an Account table. Write transfer(from_id, to_id, amount) that rolls back if the balance is too low or an account is missing. Prove the balances are unchanged after a failed transfer.

Sample run / expected result

```
transfer(1, 2, 9999) -> failed, balances still 1000 and 500
```

Hints

- Do both updates, then commit once at the end.

Problem 6: Nested Responses

Level: Challenging | **Concepts:** FastAPI + relationships + Pydantic

Add GET /authors/{id} returning the author with a list of their books, and GET /books returning each book with author name and tags.

Sample run / expected result

```
{"id": 1, "name": "Ito", "books": [{"id": 1, "title": "Tokyo Nights"}]}
```

Hints

- Nested Pydantic models with from_attributes=True

Problem 7: Docs a Teammate Can Use

Level: Stretch | **Concepts:** OpenAPI metadata

Add app title, version and description; tags with descriptions; summary, description and 404 responses on every endpoint; and examples on models. Export /openapi.json to a file.

Sample run / expected result

```
openapi.json saved, /docs shows grouped, described endpoints
```

Hints

- FastAPI(openapi_tags=[...])
- json.dump(app.openapi(), f, indent=2)

Ganbatte (頑張って): do your best